

CivicEdge: Purpose-Bound, Auditable Data Disclosure for Local-First Community Edge Nodes in Smart Cities

Devang Chaturvedi

School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology

Vellore, India

devang.chaturvedi2023@vitstudent.ac.in

Abstract—Smart-city IoT systems commonly send raw sensor readings to a remote cloud. This exposes household-level information, makes local alerts depend on wide-area connectivity, and leaves residents with little visibility into what has been shared. We present CivicEdge, a local-first community edge node that processes and stores readings within the neighbourhood and routes any external disclosure through a dedicated gateway. The gateway applies a default-deny policy tied to a stated purpose and minimum-contributor threshold. It also records releases and withholdings in a signed, hash-chained ledger that residents can verify; receipts held by the recipient provide an external anchor. We implemented CivicEdge as a self-contained container stack and ran it on a commodity home server alongside an existing self-hosted workload. In a comparison using the same live data from 111 emulated devices, median alert latency remained between 0.34 and 0.36 ms as WAN delay changed, compared with 36–365 ms for the cloud-centric baseline. CivicEdge reduced outbound data by 96.0% and sent no household identifier to the city platform. During a 90 s WAN outage, it delivered all 90 alerts within one second; the baseline delivered none in that interval. The sensors and WAN link were emulated, while the software path was real.

Keywords—*smart cities, edge computing, data minimisation, purpose limitation, tamper-evident logging, MQTT, resilience, privacy*

I. INTRODUCTION

Smart-city services rely on sensors placed throughout the places where people live: air-quality monitors, utility meters, waste-bin sensors and many others. In the common deployment pattern, readings are sent to a central cloud platform for storage, analysis and alerting [1], [2]. At neighbourhood scale, that approach creates three practical problems. Raw readings leave the community; for example, a water meter sampled once per second can reveal when a household is occupied and how its routines change. Once those readings reach an upstream service, the community has little control over their later use. Local response also depends on the wide-area network. A leak or pollution alert must travel to the cloud and back before nearby residents are notified, and may not be raised while the connection is unavailable. Finally, residents have no straightforward way to check what the system shared. A privacy policy describes a commitment, but does not by itself make that commitment verifiable.

Edge and fog computing address the second weakness by moving computation close to devices, and recent surveys cover the architectures and open problems in detail [1]–[3]. Data-space and usage-control work addresses the first by attaching machine-readable policies to shared data [9]–[11], and tamper-

evident logging work shows how to make records auditable on constrained devices [14]–[16]. These lines of work are mostly studied separately. Regulation now makes the combination practical rather than optional: India's Digital Personal Data Protection Rules, notified in November 2025, impose obligations on how personal data and the associated processing logs are held [17], and a community that operates its own sensing infrastructure needs a concrete way to show what it disclosed and why.

We present CivicEdge, a community-scale edge node built around a simple principle: keep data local unless a defined purpose justifies sharing it. The paper makes four contributions:

- A node architecture in which the component that holds raw data has no network route off the node, so that disclosure is enforced by topology as well as by code (Section III).
- A purpose-bound disclosure gateway that applies a default-deny policy with a minimum-contributor threshold and records each release and each withholding in a hash-chained, signed ledger that residents can audit and that recipient receipts anchor outside the node (Section III-D, III-E).
- A working, open implementation in about 930 lines of Python, deployed on a commodity home server next to an unrelated self-hosted workload, with no change to that workload (Section IV).
- A measured comparison with a cloud-centric baseline on the same live device fleet, covering alert latency, data leaving the node, behaviour through a WAN outage, ledger cost and resource use (Section V).

The mechanisms themselves are established techniques. In the work reviewed in Section II, however, we did not find them combined in a deployable neighbourhood node and evaluated together for both cost and performance. Our evaluation uses emulated sensors and a WAN link; Section VI explains what those results can and cannot establish.

II. RELATED WORK

A. Edge and Cloud in Smart Cities

Trigka and Dritsas survey how edge and cloud computing are combined in smart urban environments, covering frameworks, enabling technologies, applications and open challenges [1]. Belcastro *et al.* take a developer-centred view of the edge–cloud continuum and relate research models to the

platforms used in practice [2]. Bhande reviews security and privacy-preserving models for smart cities and edge environments and identifies interoperability as a recurring obstacle [3]. Chiozzotto and Ramírez compare fog computing with intelligent edge devices for smart buildings [6]. These works establish why computation should sit near devices; they are surveys and comparisons and do not prescribe what a neighbourhood node may release upstream.

B. Deployable Open-Source Platforms

Monios *et al.* describe a five-layer open-source platform for smart-city environmental monitoring, built with EdgeX Foundry, MQTT, InfluxDB and Grafana, and publish its deployment code [4]. Simões *et al.* examine the health of six open-source smart-city platforms [5]. CivicEdge shares their interest in reproducible systems built from standard components, while focusing on a different question: how should data cross from a neighbourhood to the city? Its design makes that boundary visible, restricted and auditable.

C. Messaging

MQTT is the usual transport for such systems. Lima *et al.* benchmark MQTT bridge architectures for data sharing across organisations and show that the number of bridge components and message size affect latency and reliability [7]. Gavrilidis *et al.* show that MQTT over TLS 1.3 is feasible on Raspberry Pi 4B-class hardware [8], which supports our choice to require TLS and per-device credentials even for low-cost sensors.

D. Data Sovereignty and Policy Enforcement

Amaxilatis *et al.* integrate data spaces with edge computing to share smart-building data under the owner's control [9]. Petrova-Antonova *et al.* use knowledge graphs and ODRL policy engines to enforce layered data policies in an urban data space [10], and Anthony and Sarshar study data sovereignty for mobility services [11]. Shaowang *et al.* formalise data minimisation for machine learning over IoT streams as a trade-off between accuracy and re-identification risk [12], and Borovits *et al.* systematise privacy engineering across ninety studies [13]. These approaches are expressive and aimed at federations of organisations. CivicEdge takes the opposite position on complexity: a short, human-readable policy that a resident association could read in full, enforced at a single choke point.

E. Tamper-Evident Records

Yağız *et al.* verify the integrity of IoT edge logs with Merkle trees and adaptive chunking, without a blockchain [14]. Saito combines signed sensor readouts, hash chains and blockchain-anchored Merkle trees so that authenticity survives data loss [15]. Kulothungan records AI-driven IoT decisions on a permissioned blockchain for audit [16]. CivicEdge applies the same primitives to a different object: the log covers *disclosure decisions* instead of sensor readings or model outputs, including decisions to withhold, and it is anchored by the receipts the recipient already holds, so no blockchain is needed.

III. SYSTEM DESIGN

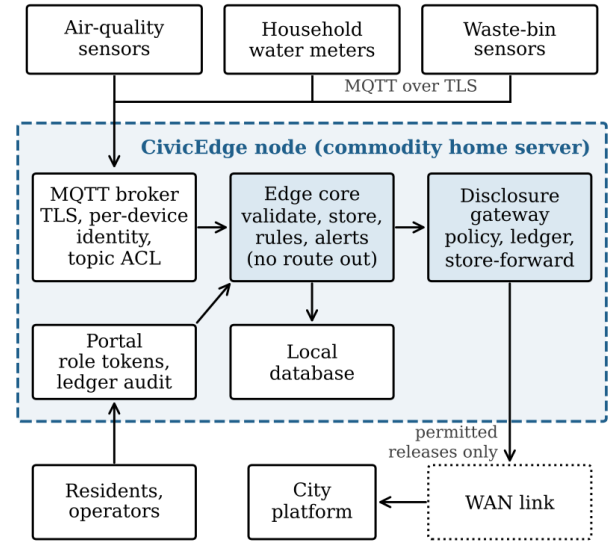
A. Goals and Threat Model

CivicEdge has four design goals: (G1) keep raw and household-level data on the node unless a written policy allows its release; (G2) keep local alerting independent of the WAN; (G3) let residents verify what was shared; and (G4) run on

hardware that a housing society or ward office could own. Our threat model assumes that the operator processes data honestly but could later face pressure or an incentive to conceal a disclosure. A person on the local network may spoof devices. The city platform is treated as a legitimate recipient, but it should receive no more than the policy permits. We do not address physical compromise of the node or side-channel attacks.

B. Architecture

Fig. 1 shows the node. Devices publish to an MQTT broker. The *edge core* subscribes, validates and stores readings, evaluates rules and raises alerts. The *disclosure gateway* is the only component connected to the uplink. A *portal* is the single published port for residents and operators. The core is attached only to node-internal networks that have no gateway to the outside, so a programming error in the core cannot send data upstream: there is no route. This separation is what turns the policy from a convention into a property of the deployment.



1. CivicEdge node architecture. Only the disclosure gateway is attached to the uplink network.

C. Device Ingest

Each device has its own broker identity and connects over TLS. The broker's access-control list lets a device publish only to its own topic, and only the core may subscribe. The core additionally rejects a payload whose claimed identity differs from its topic, and applies schema and range checks. A reading that fails any check is counted and dropped.

D. Purpose-Bound Disclosure Policy

The disclosure rules are held in a short YAML file (Table I). The policy denies disclosure by default: any stream or event type without an explicit entry stays on the node. For an allowed stream, the gateway sends only a per-zone aggregate over a defined time window, and only when at least k distinct devices contributed. It withholds groups below that threshold. Allowed events are reduced to incidents containing only approved fields. Household leak alerts are deliberately absent from the policy; residents receive them locally, but the gateway never forwards

them. Each ledger entry includes the policy file's SHA-256 digest, making later policy changes visible during an audit.

I. DISCLOSURE POLICY USED IN THE EVALUATION

Data	Purpose	Released form	Condition
Air quality	Public-health reporting	Zone mean, max, count per 30 s	$k \geq 3$ devices
Household water	Utility demand planning	Zone mean, count per 30 s	$k \geq 5$ households
Air exceedance	Emergency response	Zone, severity, value	On alert
Bin full	Collection routing	Bin ID, zone	On alert
All else	None	Never released	Default deny

E. Disclosure Ledger

Every decision of the gateway becomes one ledger entry e_i holding the sequence number, time, decision (release or suppress), stream, purpose, recipient, policy digest, window, number of contributing devices, and the SHA-256 digest and size of the payload. Entries are chained: $h_i = \text{SHA-256}(c(e_i) \parallel h_{i-1})$, where c is a canonical JSON encoding and h_0 is a fixed constant. The node signs each h_i with an Ed25519 key created at first start. Editing, deleting or reordering any entry breaks the chain or a signature. Withheld items are logged without content, so the ledger shows that the threshold was applied without revealing the small group's values.

A hash chain by itself cannot prevent an operator from rebuilding the ledger using the same signing key. CivicEdge addresses this by sending a signed entry with every release. The recipient verifies the signature and keeps (i, h_i) as a receipt. Since each hash covers all preceding entries, that receipt anchors the ledger prefix, including withholding decisions that were never sent to the recipient. An auditor can compare the node's ledger with the saved receipts and detect changes to any history covered by the latest receipt.

F. Store-and-Forward

Releases are queued in the ledger and sent strictly in order. When the uplink fails, the gateway retries; windows continue to close and to be recorded, and the queue drains when the link returns. Alerting is unaffected because it happens in the core.

IV. IMPLEMENTATION

CivicEdge is implemented in Python 3.12, with about 930 lines covering the node, recipient stand-in and emulators. The Docker Compose project uses one image. Eclipse Mosquitto provides the broker; SQLite stores readings and ledger entries; FastAPI serves the HTTP interfaces; and the cryptography library handles X.509 certificates and Ed25519 signatures. On first start, an initialisation job creates the node certificate authority, broker certificate, per-device broker identities and signing key. Five Docker networks separate devices, node internals, the uplink, recipient and published portal; four have no external connectivity.

The portal exposes alerts, zone summaries, the ledger and a verification endpoint, with separate resident and operator tokens. Residents see community alerts but never another household's alert or any device identifier. A command-line auditor re-derives the chain, checks every signature and cross-checks recipient receipts from any machine that can reach the portal.

We deployed CivicEdge on JellsTrip, a home server that already hosts media, photo, DNS, VPN and identity services in a Proxmox LXC container. CivicEdge runs as a separate Compose project with its own networks and storage volumes. The deployment required no edits to the existing configuration, and it can be removed with one command. Administration uses the host's existing Tailscale network. An optional notifier can also post community alerts to a Matrix room on the existing homeserver. This setup reflects the shared, modest hardware we expect a neighbourhood group to use, rather than a dedicated appliance.

A stand-in city platform verifies disclosures and keeps receipts. For evaluation the same gateway has a second mode, the *cloud-centric baseline*, in which every raw reading is forwarded upstream in batches, the recipient evaluates the same rules with the same code, and alerts are returned to the node. The baseline keeps up to four batches in flight so that it is not penalised by a serialised uplink, and it buffers during outages so that no reading is lost.

V. EVALUATION

A. Setup

All experiments ran on the deployment host (Intel(R) Core(TM) i3-10100F CPU @ 3.60GHz, a quad-core desktop processor; the container is allocated 4 cores and 12 GiB of memory, shared with the existing workload of 66 other containers). The fleet has 111 emulated devices in six zones: 27 air-quality sensors, 54 household water meters and 30 waste-bin sensors. Each is a separate MQTT client with its own credentials and TLS session, publishing once per second, about 111 messages per second in total. One zone is deliberately sparse (two air sensors, four meters) to exercise the threshold. Exceedance episodes are injected at one per second across the fleet so that alerts are frequent enough to measure; this rate is far above what a real neighbourhood would see and makes the data-volume results conservative for CivicEdge.

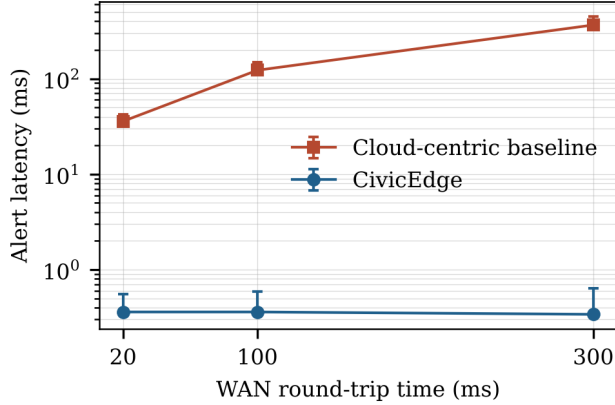
The WAN link is an in-path relay that adds a configurable one-way delay with $\pm 10\%$ jitter and can be switched off. Alert latency is the time from the timestamp of the reading that completes a rule to the moment the alert is recorded on the node; all components share the host clock. Each configuration ran for 90 s after a settling period. Both modes ran on the same live fleet, one after the other.

B. Alert Latency

Table II and Fig. 2 show alert latency for three WAN round-trip times (537 alerts in total). The baseline's latency follows the round-trip time, with a median from 35.8 ms at 20 ms to 365.1 ms at 300 ms. CivicEdge's median stays between 0.34 and 0.36 ms in every configuration, with a worst observed value of 2.9 ms, because the rule is evaluated where the reading arrives. The difference is expected; the measurement fixes its size on shared commodity hardware and confirms that local evaluation is unaffected by the state of the uplink.

II. ALERT LATENCY (MS) AGAINST WAN ROUND-TRIP TIME

WAN RTT	Baseline p50	Baseline p95	CivicEdge p50	CivicEdge p95
20 ms	35.8	42.3	0.36	0.55
100 ms	122.5	149.5	0.36	0.59
300 ms	365.1	445.2	0.34	0.64



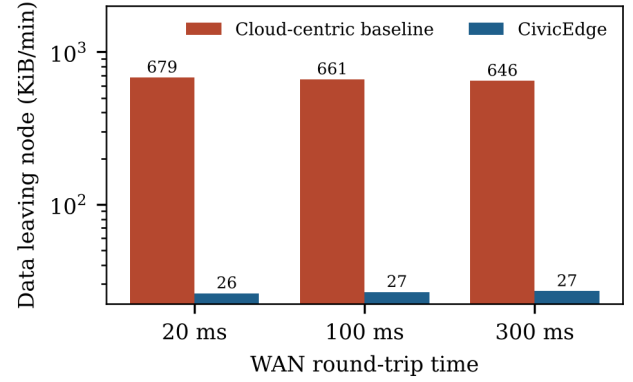
- Alert latency against WAN round-trip time. Markers show the median; bars extend to the 95th percentile.

C. Data Leaving the Node

Table III and Fig. 3 compare what crosses the uplink. The baseline sends 662 KiB/min, essentially everything ingested (631 KiB/min of sensor payload plus batching overhead). CivicEdge sends 26.5 KiB/min, a reduction of 96.0% (25.0 times less), most of which is the signed ledger entry that accompanies each release. More important than volume is content. Under the baseline the recipient received readings from all 111 devices, including 5,075 per-second readings from 54 identifiable households in a single 90 s run. Under CivicEdge it received no household reading and no household identifier; the only device identifiers it saw were those of full waste bins, which the policy releases for collection routing.

III. DATA CROSSING THE UPLINK (MEAN OF THREE RUNS; COUNTS FROM ONE 90 S RUN)

Metric	Baseline	CivicEdge
Data sent upstream (KiB/min)	662	26.5
Device-level records sent (per min)	6,660	15.3
Distinct device IDs seen by city	111	18
Households identifiable at city	54	0
Household readings held by city	5,075	0
Items withheld by policy	n/a	56



- Data leaving the node per minute (logarithmic scale).

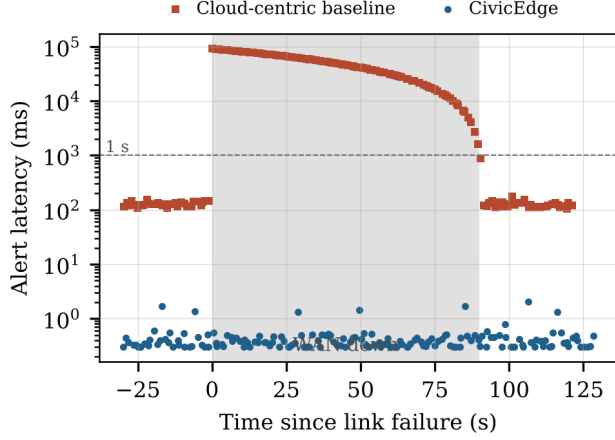
D. WAN Outage

The link was cut for 90 s at a 100 ms round-trip time (Fig. 4, Table IV). Under the baseline, 90 alert conditions arose during the outage and 0 were delivered within one second; they arrived after the link returned, with a median delay of 46.1 s and a maximum of 90.4 s. Under CivicEdge all 90 alerts raised during the outage were delivered locally, 90 of them within one second, with a median of 0.38 ms, the same as with the link up.

When the link returned, the baseline had 10,006 raw readings queued and CivicEdge had 56 signed releases queued; these drained in 1.2 s and 7.8 s respectively. CivicEdge drains more slowly because releases are sent one at a time in ledger order, each waiting a full round trip; the baseline sends large batches. Nothing was lost in either mode: the city received all 94 CivicEdge releases produced in the run, and none remained pending.

IV. BEHAVIOUR THROUGH A 90 S WAN OUTAGE

Metric	Baseline	CivicEdge
Alert conditions during outage	90	90
Delivered within 1 s	0	90
Median alert delay during outage	46.1 s	0.38 ms
Queue at link restore	10,006 readings	56 releases
Time to drain queue	1.2 s	7.8 s



4. Alert latency before, during and after a WAN outage (shaded). Each point is one alert.

E. Ledger Integrity and Cost

At the end of the experiments the ledger held 562 entries (281 releases and 281 withholdings). An independent audit re-derived the chain, verified every signature, and matched all 281 recipient receipts with 0 mismatches. In a separate benchmark on the same host, appending an entry (hash, signature and durable write) took 0.54–1.27 ms and verification took 119 μ s per entry, so a ledger of 50,000 entries verifies in 6.0 s. Four kinds of tampering were applied to a 2,000-entry ledger, 250 random trials each: changing a field, deleting an entry, swapping two entries, and rewriting an entry with a recomputed hash signed by a different key. Verification detected 1,000 of 1,000.

F. Resource Use

Table V gives the footprint sampled every few seconds across the whole evaluation. The four node services together used 8.6% of one core on average and 187 MiB of memory. The device emulator, which stands in for 111 separate devices, is the largest consumer and would not exist in a real deployment.

V. RESOURCE USE BY SERVICE (* EVALUATION HARNESS, NOT PART OF THE NODE)

Service	CPU mean (%)	CPU max (%)	Memory mean (MiB)
MQTT broker	0.79	1.2	8
Edge core	4.02	14.2	100
Disclosure gateway	3.76	17.5	44
Portal	0.07	0.1	34
City stand-in*	1.73	7.7	40
WAN relay*	3.59	17.4	38
Device emulator*	2.26	3.4	40

VI.

DISCUSSION AND LIMITATIONS

The latency and outage results are a direct consequence of running alert rules locally. We include them to show how that design performs on a shared home server and what the disclosure controls cost: the policy gateway and signed ledger add about a millisecond per decision and less than a kilobyte per release. In this setup, those measurements suggest that purpose-bound, auditable disclosure is practical at neighbourhood scale.

The evaluation has several limits. We emulated the sensors, so the real software path from TLS handshake to ledger does not capture radio loss, clock drift, device failure or calibration errors. Synthetic traces cannot substitute for field data. We also emulated the WAN with an in-path relay, fixed delay and a clean disconnect; real links tend to degrade less predictably. We tested one node on one host, with the recipient on that same machine, so these results do not establish how multiple nodes or a municipal platform would perform. Our baseline is a reference implementation; a tuned commercial cloud pipeline may behave differently, though it would still require a WAN round trip and would not deliver alerts during an outage. The contributor threshold also does not provide a formal privacy guarantee. Releasing zone means based on at least k devices limits exposure, but it is not differential privacy, and repeated windows may reveal more than a single window. External anchoring relies on the recipient retaining receipts; entries after the latest receipt are protected only by the chain and the node's key. We leave encryption at rest to the host's volume configuration. Finally, the injected event rate increases CivicEdge's upstream traffic, so the reported reduction is a lower bound for this configuration.

Next, we plan to repeat the evaluation with physical sensors, publish ledger heads to an independent witness, test calibrated noise for aggregates, and connect multiple nodes to a shared recipient.

VII.

CONCLUSION

Our results show that a neighbourhood node can keep sensor data local, raise alerts without WAN access, and still provide the city with policy-approved information using commodity hardware and open-source components. CivicEdge makes the community–city boundary concrete: a concise policy defines what may be shared and for which purpose, network topology prevents raw data from routing out, and a recipient-anchored signed ledger lets residents audit disclosure decisions. In our deployment with 111 emulated devices, median alert latency remained below one millisecond regardless of WAN conditions, outbound data fell by 96.0%, and no household identifier left the node. Both local alerting and queued disclosure continued through a WAN outage without data loss. The implementation, policy and experiment scripts are available to support reproduction of these results.

REFERENCES

1. M. Trigka and E. Dritsas, "Edge and cloud computing in smart cities," *Future Internet*, vol. 17, no. 3, Art. no. 118, 2025, doi: 10.3390/fi17030118.
2. L. Belcastro, F. Marozzo, A. Orsino, D. Talia, and P. Trunfio, "Navigating the edge-cloud continuum: A state-of-practice survey," arXiv:2506.02003, 2025.
3. A. Bhande, "A systematic review of data security and privacy-preserving models for smart cities and edge computing environments," *EPJ Web*

Conf., vol. 341, Art. no. 01024, 2025, doi: 10.1051/epjconf/202534101024.

4. N. Monios, P. Papageorgas, D. Piromalis, V. Cheimaras, and G. Sarigiannis, "A five-layer open-source IoT platform architecture for smart city applications," *Electronics*, vol. 15, no. 15, Art. no. 3377, 2026, doi: 10.3390/electronics15153377.
5. R. B. Simões, F. Brito e Abreu, and V. Amaral, "Evaluating the health of open-source smart city platforms," arXiv:2609.18801, 2026.
6. M. Chiozzotto and M. A. Ramírez, "What is the best solution for smart buildings? A case study of fog, edge computing and smart IoT devices," *Appl. Sci.*, vol. 15, no. 7, Art. no. 3805, 2025, doi: 10.3390/app15073805.
7. K. Lima, T. D. Oyetoyan, R. Heldal, and W. Hasselbring, "Evaluation of MQTT bridge architectures in a cross-organizational context," arXiv:2501.14890, 2025.
8. N. O. Gavrilidis, S. T. Halkidis, and S. Petridou, "Empirical evaluation of TLS-enhanced MQTT on IoT devices for V2X use cases," *Appl. Sci.*, vol. 15, no. 15, Art. no. 8398, 2025, doi: 10.3390/app15158398.
9. D. Amaxilatis, T. Sarantakos, N. Tsironis, S. Sengupta, K. Ramantas, and J. Ojeda, "Harnessing data spaces to build intelligent smart city infrastructures across the cloud-edge continuum," arXiv:2512.21340, 2025.
10. D. Petrova-Antonova *et al.*, "Knowledge graph-driven policy enforcement in urban dataspace," in *Knowledge Graphs (IJCKG 2025)*, Lecture Notes in Computer Science, vol. 16297. Singapore: Springer, 2026, pp. 406–421, doi: 10.1007/978-981-95-5009-8_27.
11. B. Anthony and S. Sarshar, "Enhancing data sovereignty to improve intelligent mobility services in smart cities," *Urban Governance*, 2025, doi: 10.1016/j.ugj.2025.02.002.
12. T. Shaowang, S. Liu, J. Marques, N. Feamster, and S. Krishnan, "Algorithmic data minimization for machine learning over Internet-of-Things data streams," arXiv:2503.05675, 2025.
13. N. Borovits, D. A. Tamburri, and W.-J. Van Den Heuvel, "Privacy engineering: A systematic literature review," arXiv:2606.23696, 2026.
14. M. A. Yağız, F. Horasan, and A. H. Yurttakal, "Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle-tree pipeline with adaptive chunking," arXiv:2605.00065, 2026.
15. K. Saito, "Proof of authenticity of general IoT information with tamper-evident sensors and blockchain," arXiv:2512.18560, 2025.
16. V. Kulothungan, "Using blockchain ledgers to record AI decisions in IoT," *IoT*, vol. 6, no. 3, Art. no. 37, 2025, doi: 10.3390/iot6030037.
17. Ministry of Electronics and Information Technology, Government of India, "Digital Personal Data Protection Rules, 2025," Notification G.S.R. 846(E), *Gazette of India*, Nov. 13, 2025.